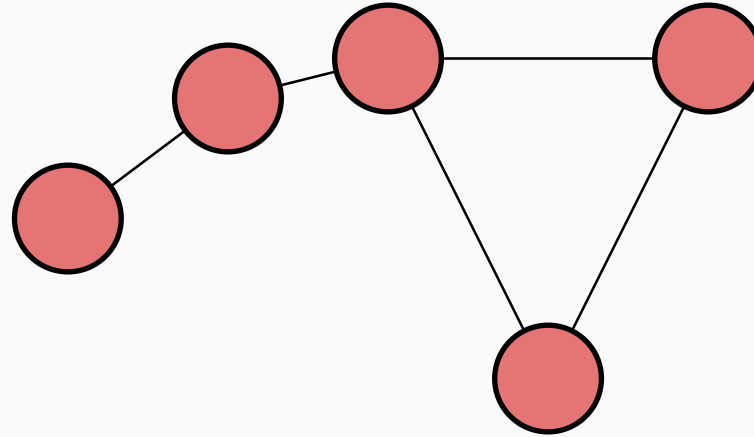


Introduction aux graphes

1. Introduction

1.1 C'est quoi ?

Un graphe c'est quoi ?



Definition 1.1.1 (Graphe)

Un graphe est un ensemble de noeuds connectés par des arêtes. On dénote le graphe comme un couple (V, E) , où V est l'ensemble des noeuds, et E l'ensemble des arêtes.

V pour “**V**ertices” et E pour “**E**dgés” ...

2. Quelques définitions

2.1 Connexité

Definition 2.1.1 (Connexité)

On dit qu'un graphe est **connexe** si et seulement si il existe un chemin entre n'importe quel noeud a et b .

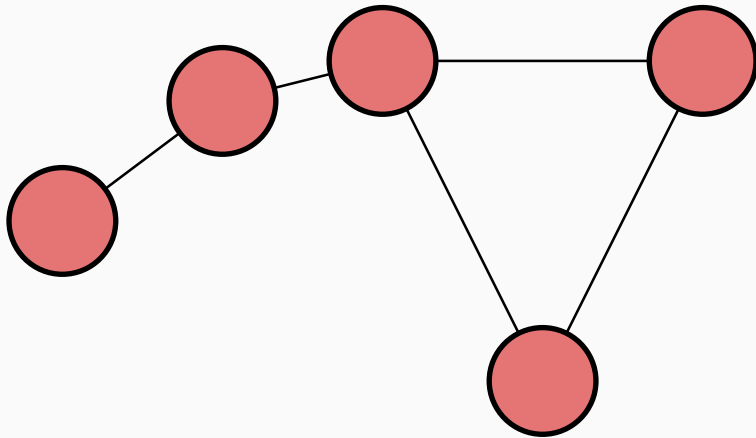


Figure 1: Graphe connexe

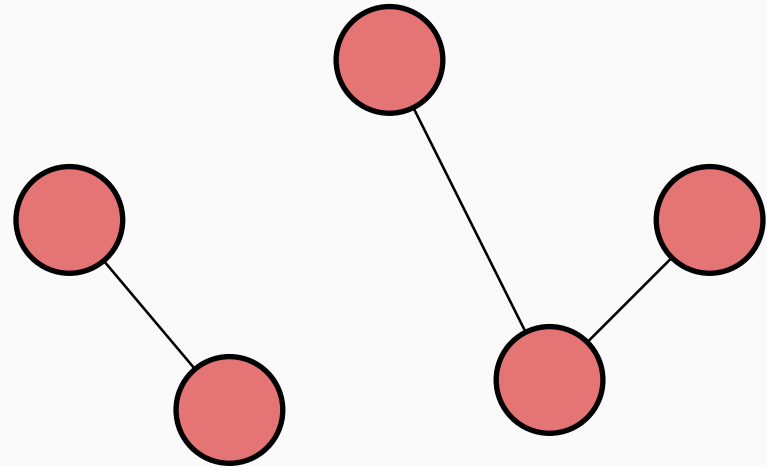


Figure 2: Graphe non connexe

2.1 Connexité

De plus, les parties **connexes** d'un graphe sont appelées les **composantes connexes**.

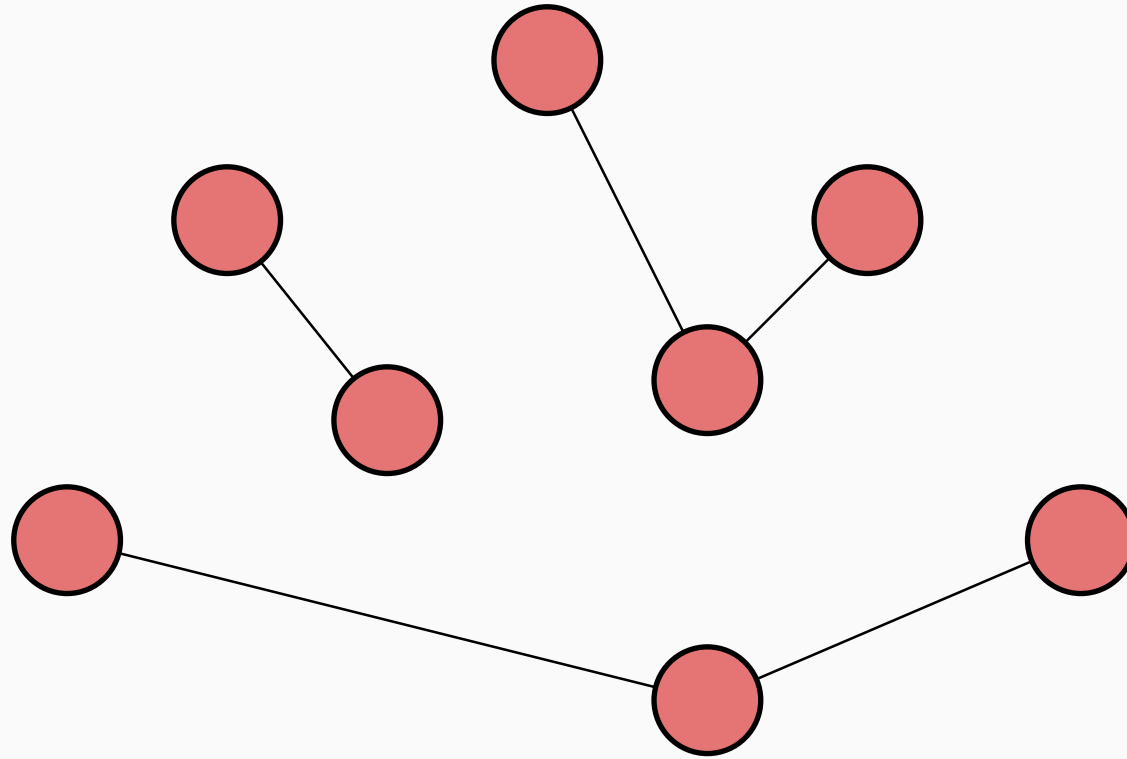


Figure 3: Ce graphe a 3 composantes connexes

2.2 Graphe orienté

Definition 2.2.1 (Graphe orienté)

Un graphe est **orienté** si et seulement si pour chaque arête, on peut se déplacer dans un seul sens.

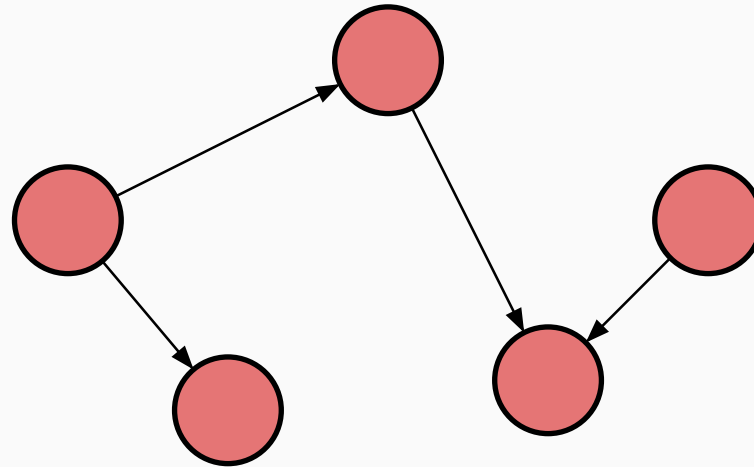


Figure 4: Les flèches représentent le sens de déplacement

2.3 Voisinage

Definition 2.3.1 (Voisin)

On dit que b est un **voisin** de a s'il y a possibilité d'aller de a vers b .

Definition 2.3.2 (Ancêtre)

Dans le cas d'un graphe orienté, a est un **ancêtre** de b s'il y a possibilité d'aller de a vers b .

Remarques:

- Du point de vue de a , b sera son voisin.
- Dans un graphe non-orienté, la notion d'ancêtre et de voisin est confondu.

Definition 2.3.3 (Degré)

Le **degré** d'un noeud a est le nombre de liens reliant ce sommet.

Dans le cas d'un graphe orienté, on parle de **degré entrant**, le nombre d'arêtes dirigées vers le noeud. Dans le cas contraire, on parle de **degré sortant**.

2.4 Graphe pondéré

Definition 2.4.1 (Graphe pondéré)

Un graphe est **pondéré** si et seulement si chaque arête lui est attribué une valeur.

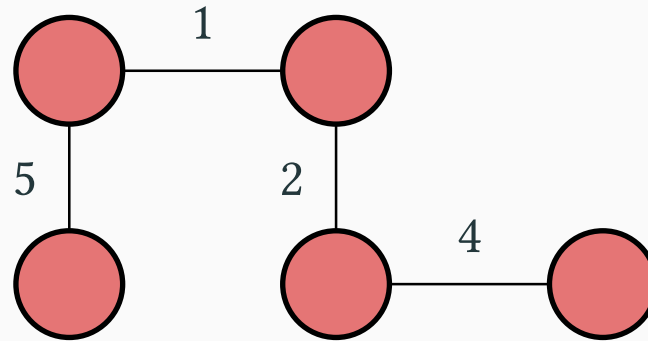


Figure 5: Graphe pondéré

*Remarque : Les valeurs sur les arêtes sont les **poids** des arêtes ...*

3. Représentation d'un graphe

3.1 Matrice d'adjacence

Soit $G = (V, E)$, on peut représenter un graphe sous la forme d'une matrice carré $M = (a_{i,j})$ de taille $|V|$, où :

$$\forall i, j \in \llbracket 1, |E| \rrbracket, a_{i,j} = 1 \text{ si } (i, j) \in E \text{ sinon } a_{i,j} = 0$$

Note

Si G est non-orienté, alors M est symétrique.

Note

Dans le cas d'un graphe pondéré, la valeur 1 est remplacée par le poids de l'arête.

3.2 Liste d'adjacence

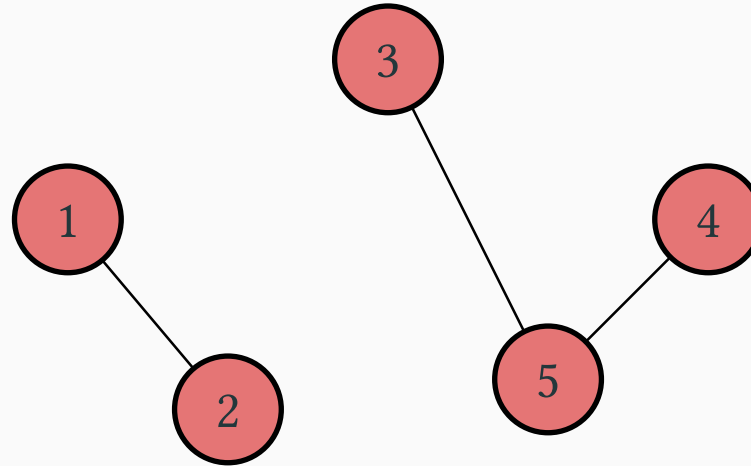
Soit $G = (V, E)$, on peut représenter un graphe sous la forme d'une liste A de taille $|V|$:

$$\forall a \in V, A[a] = [a_1, a_2, \dots, a_k]$$

où $a_1, a_2, \dots, a_k$ représentent les noeuds vers lesquels nous pouvons aller depuis a .

Nous pouvons aller plus loin en termes d'optimisation de mémoire, en remplaçant A par un dictionnaire D . L'avantage ici c'est que nous stockons uniquement les noeuds qui ont des voisins.

3.3 Exemple



Ce qui donne :

$$\begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{pmatrix}$$

Figure 6: Matrice d'adjacence

$$A = [[2], [1], [5], [5], [3, 4]]$$

Figure 7: Liste d'adjacence

$$A = \{1 : [2], 2 : [1], 3 : [5], 4 : [5], 5 : [3, 4]\}$$

Figure 8: Liste d'adjacence
(dictionnaire)

3.4 En pratique

Dans la pratique, les problèmes sont posés de tel sorte à ce que, lorsqu'il y a $|V|$ noeuds, ils sont numérotés dans l'ordre de 1 jusqu'à $|V|$.

Note

Il est ainsi peu intéressant d'utiliser la version “dictionnaire” d'une liste d'adjacence, surtout à cause des problèmes de hachage qui peuvent survenir.

Note

La version matricielle a un usage limité, car en termes de mémoire, nous avons souvent affaire à des graphes avec $|V| \approx 10^5$. Nous verrons dans le futur comment utiliser cette représentation pour passer dans le domaine algébrique.

4. Parcours

4.1 Motivations

Lorsque nous avons un graphe, en général la première chose que nous souhaitons faire c'est de pouvoir trouver un chemin entre 2 noeuds.

On distingue 2 types de recherche :

- **Parcours en profondeur** (en anglais *depth first search* qu'on nomme *DFS*)
- **Parcours en largeur** (en anglais *breadth first search* qu'on nomme *BFS*)

⚠ Attention

Par la suite, nous supposons que nous travaillons avec un graphe représenté sous la forme d'une liste d'adjacence. Ce qui donne en Python `list[list[int]]`.

📄 Note

Un petit tour rapide sur les piles et files en Python serait envisageable d'abord ...

4.2 Parcours en profondeur *depth first search*

L'idée d'un parcours en profondeur, c'est que pour chaque noeud:

- On se déplace vers le premier non visité.
- On marque le noeud comme "visité".

Il suffit donc de commencer la recherche à partir d'un noeud de départ a , et de s'arrêter lorsque nous atteignons l'arrivée b .

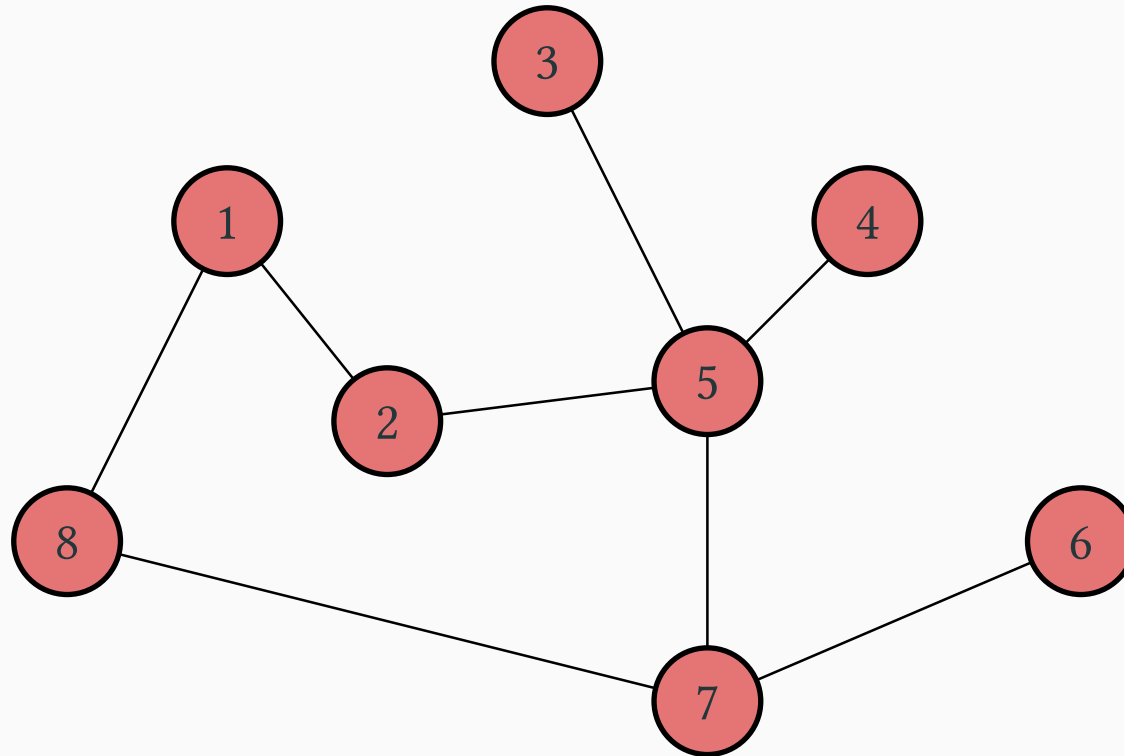
Voici une version récursive de l'algorithme de recherche :

```
def dfs(graph : list[list[int]], nfrom : int, nto : int, visited : set) -> bool:
    if nfrom in visited:
        return False
    visited.add(nfrom)
    result = False
    for neighbour in graph[nfrom]:
        if neighbour not in visited:
            result = result or dfs(graph, neighbour, nto, visited)
    return result
```

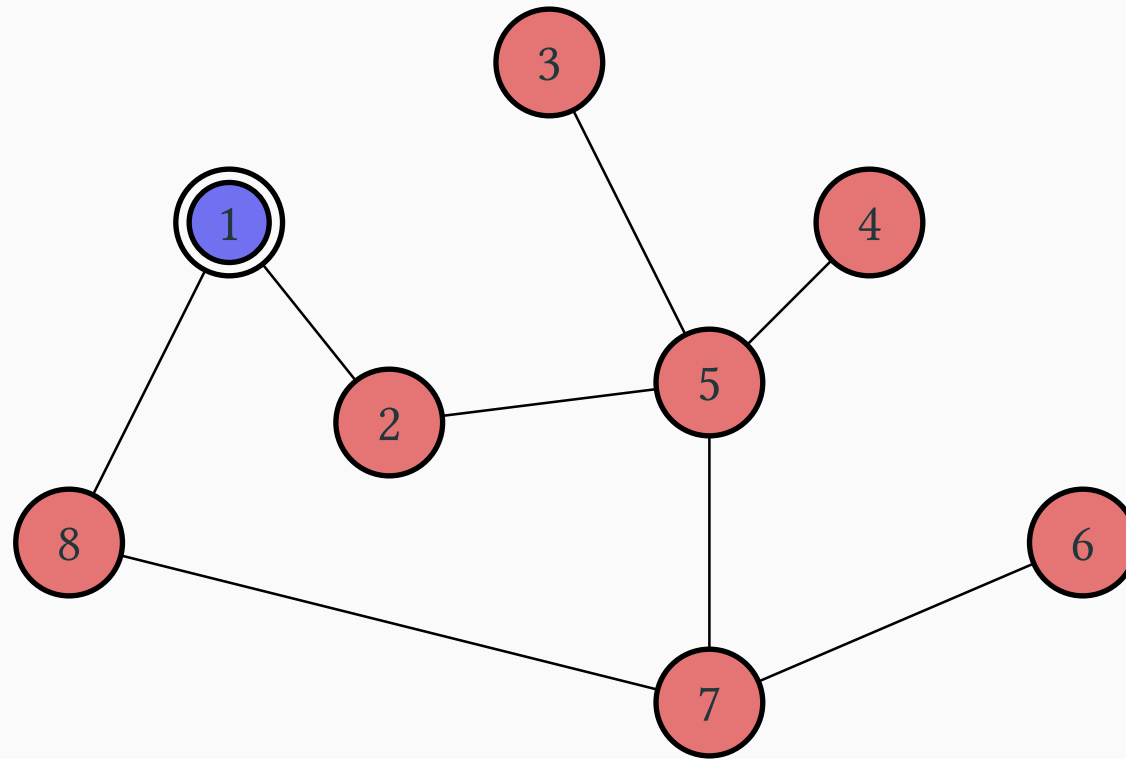
4.3 Exemple *BFS*

Les noeuds visités seront colorés en bleu, et le noeud où nous nous trouvons aura une double bordure.

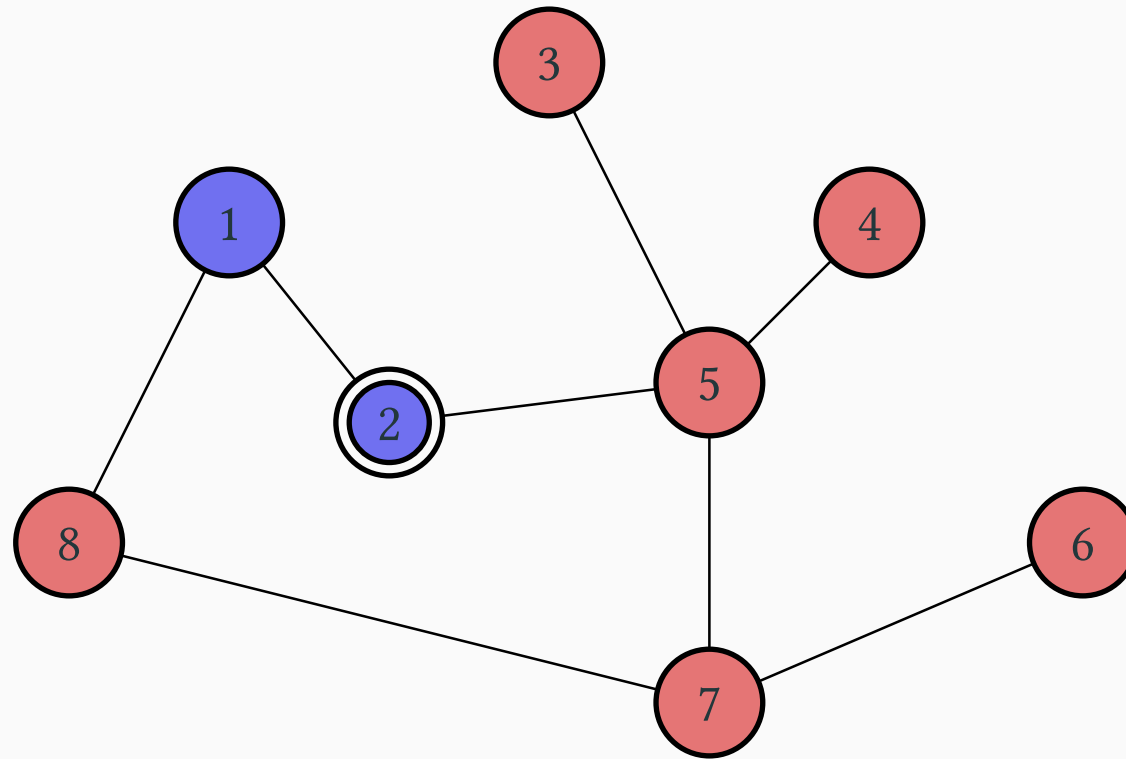
On souhaite aller du noeud 1 au noeud 6.



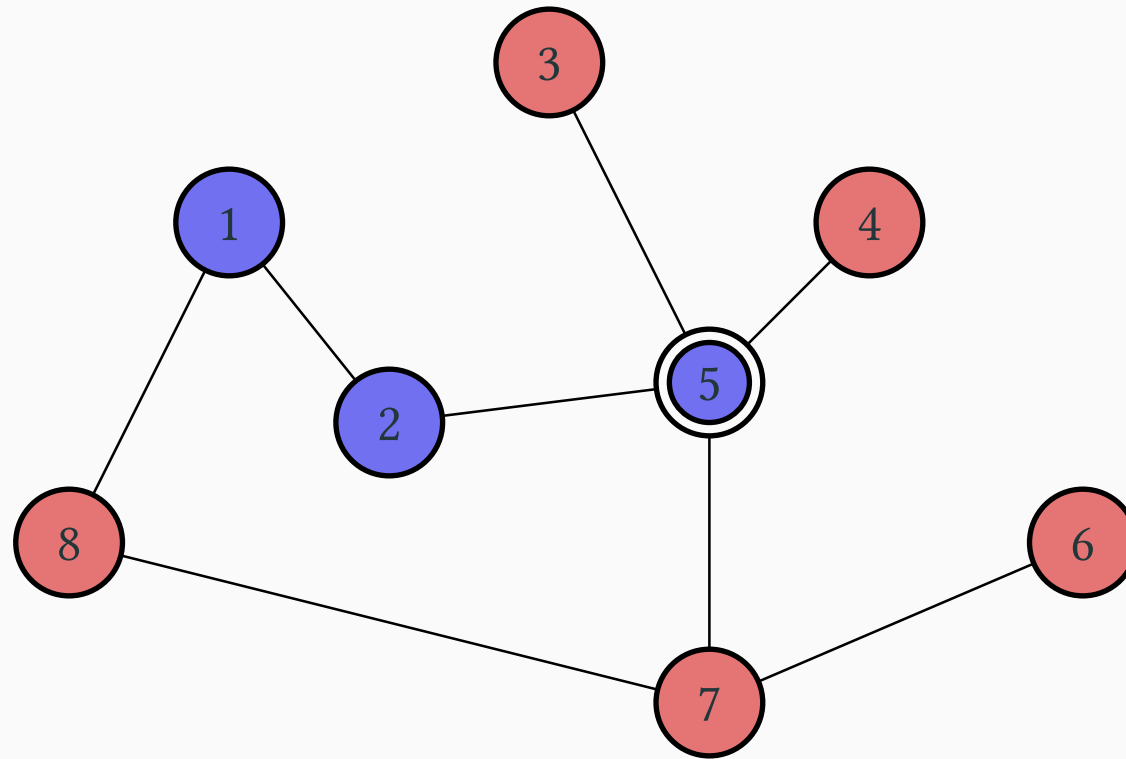
4.3 Exemple *BFS*



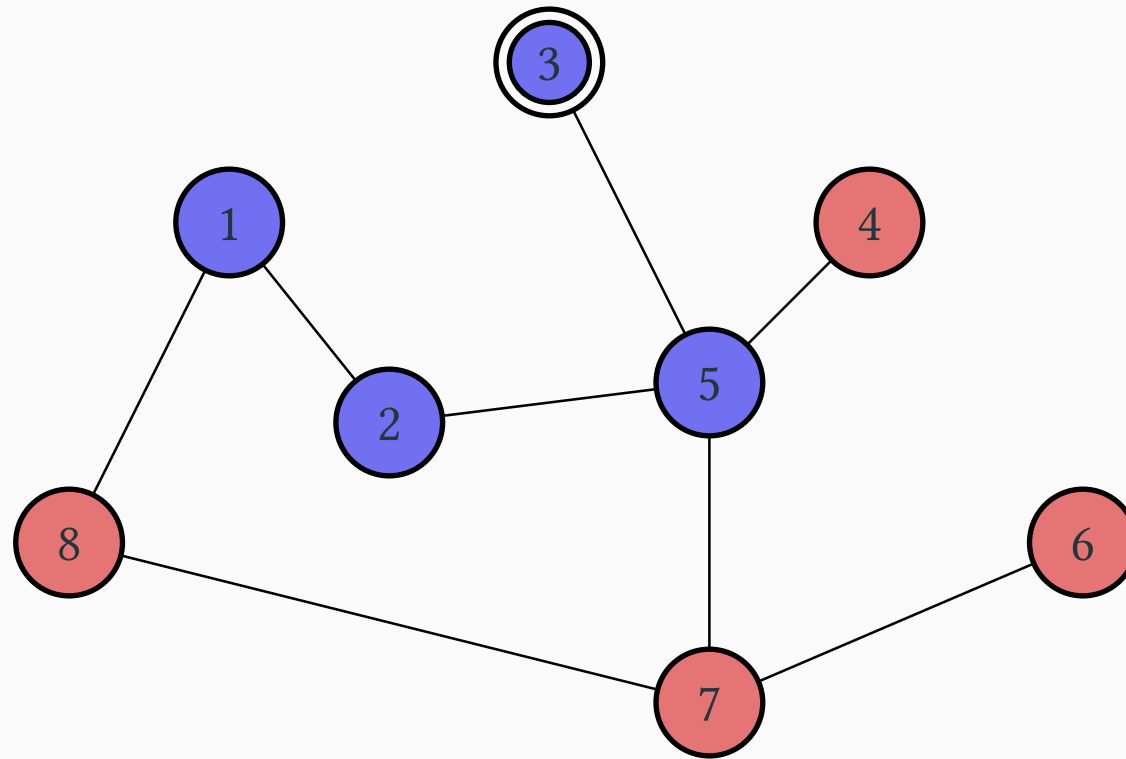
4.3 Exemple *BFS*



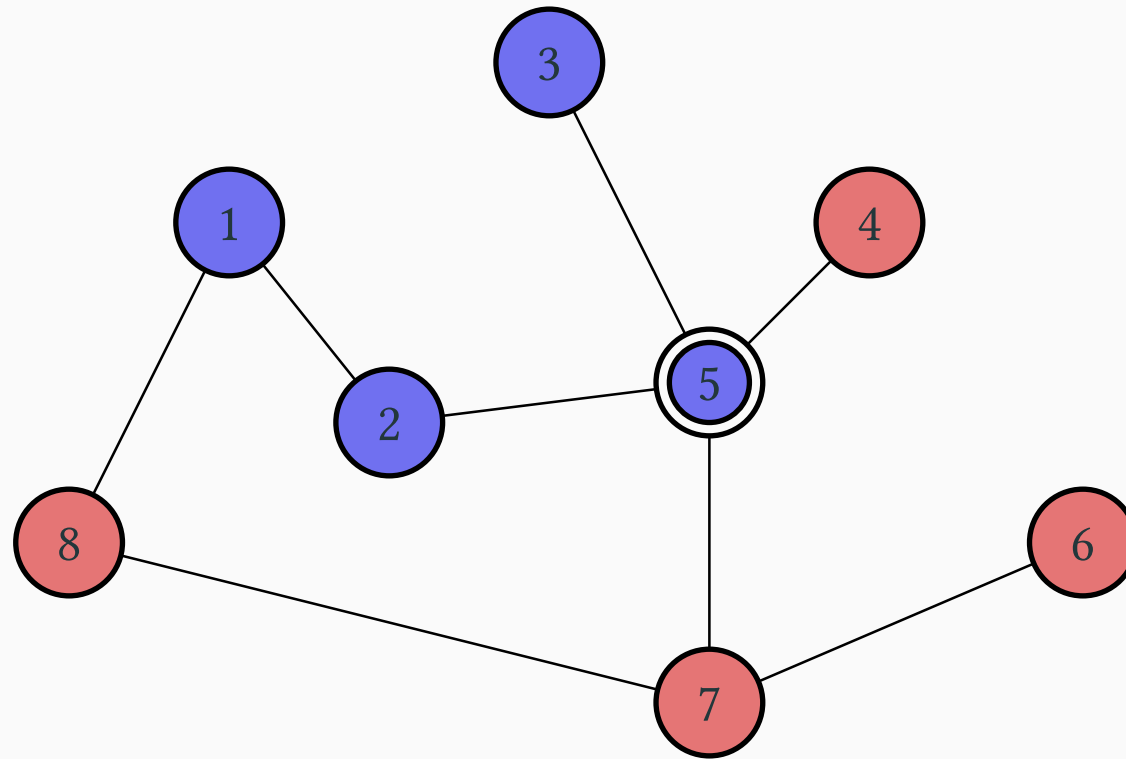
4.3 Exemple *BFS*



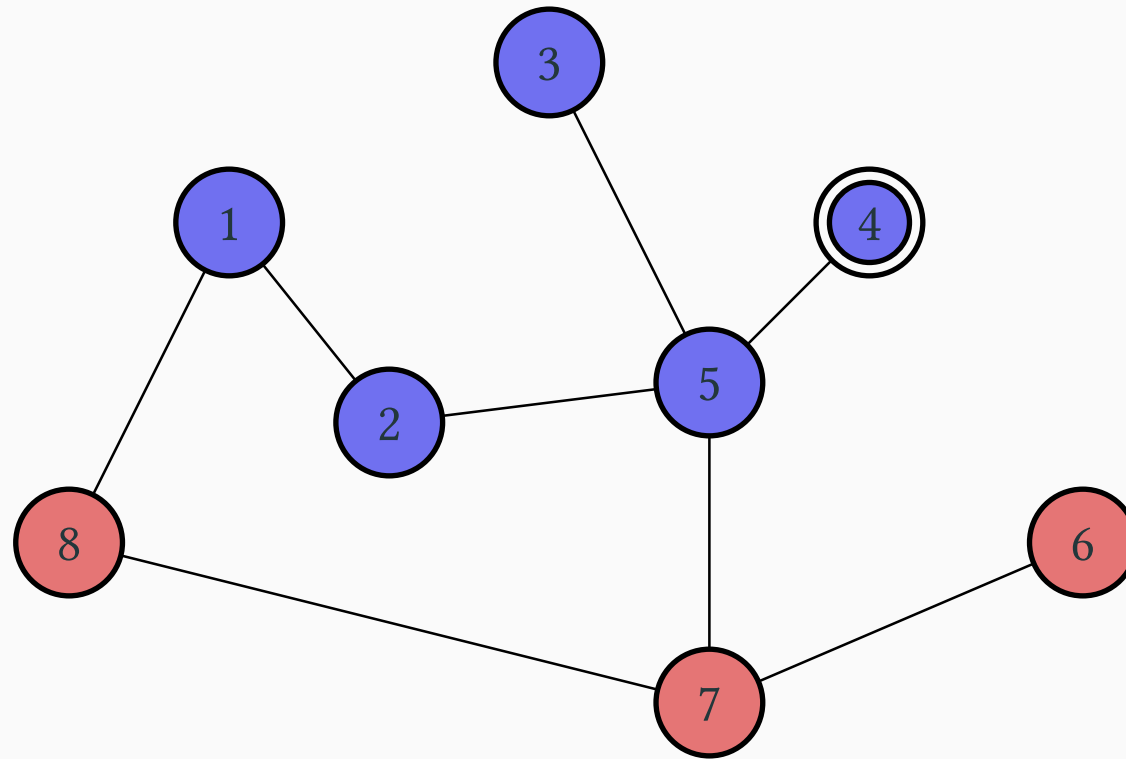
4.3 Exemple *BFS*



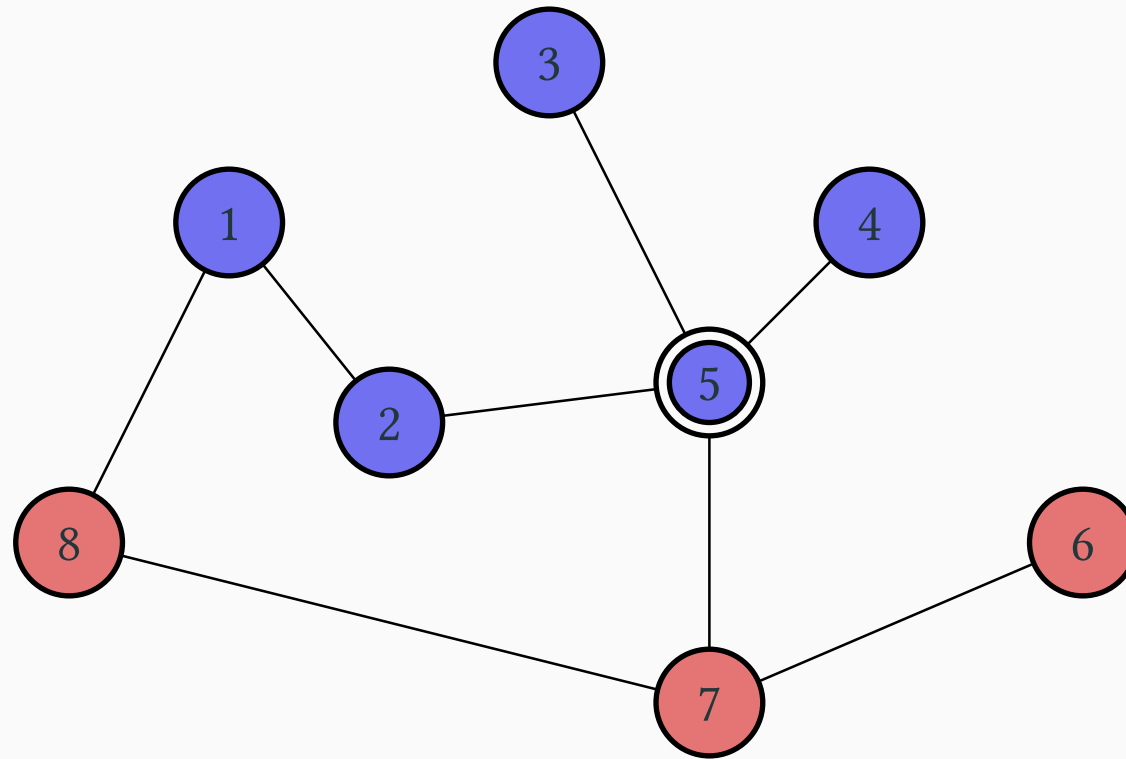
4.3 Exemple *BFS*



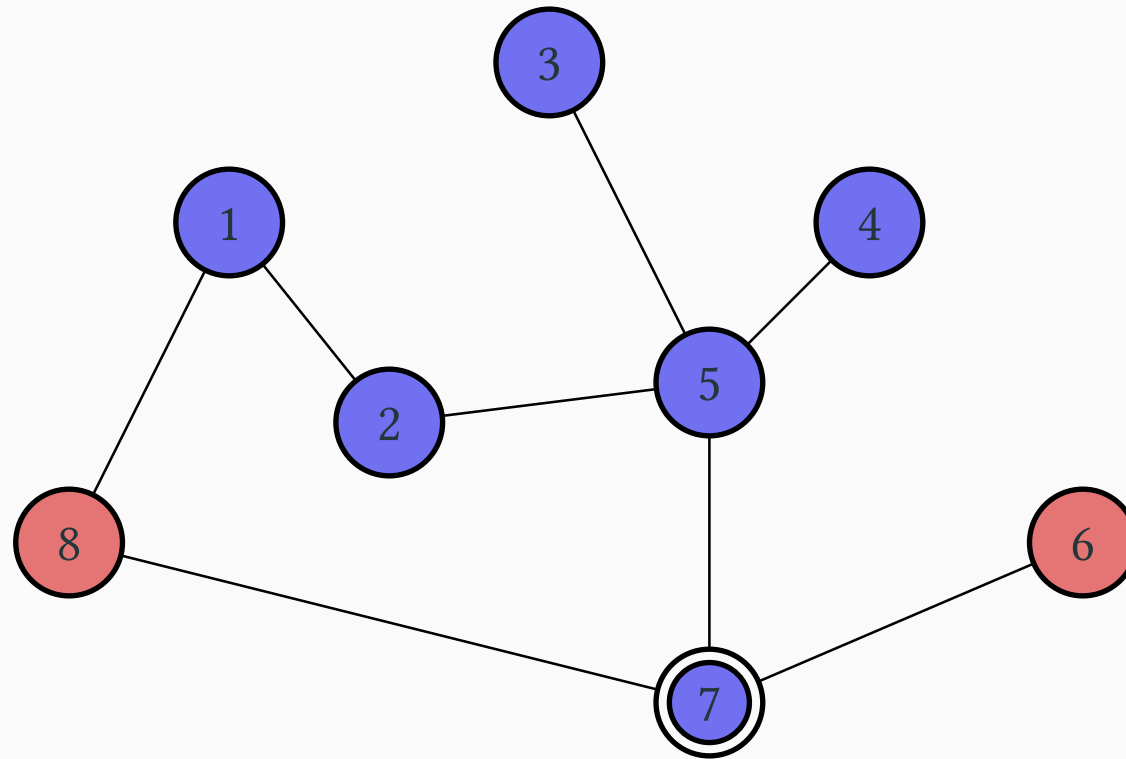
4.3 Exemple *BFS*



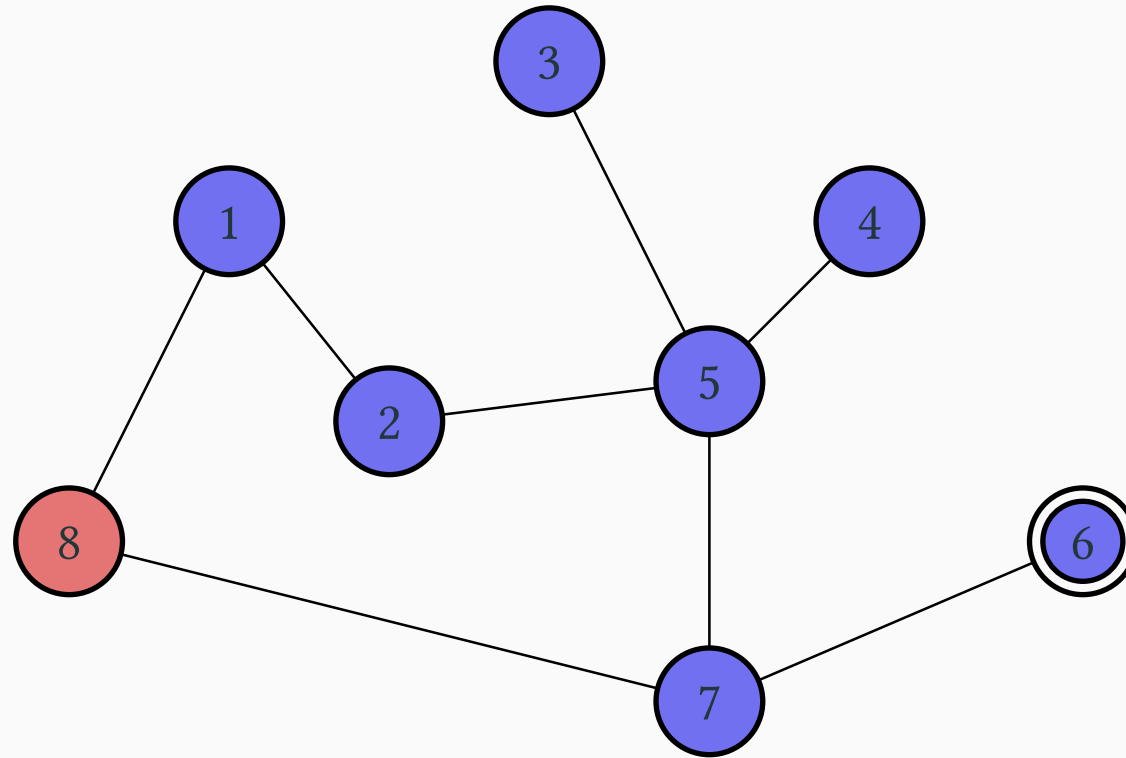
4.3 Exemple *BFS*



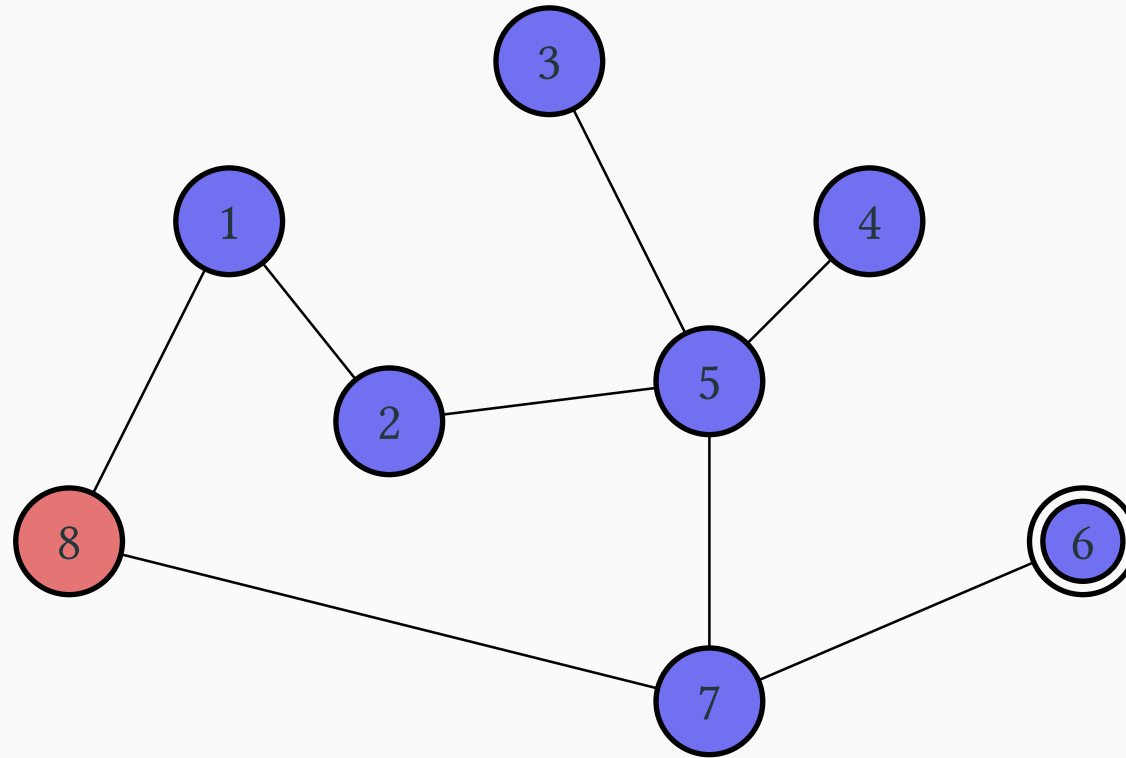
4.3 Exemple *BFS*



4.4 Exemple *BFS*



4.4 Exemple *BFS*



Nous avons trouver le noeud 6, il existe bien un chemin entre 1 et 6 !

4.5 Méthode itérative

En Python, étant donné la lenteur de la récursivité, il y a plus intérêt d'utiliser une méthode itérative.

Pour cela, il nous suffit de simuler la pile d'appel nous même.

Voici un exemple en Python :

```
def dfs(graph : list[list[int]], nfrom : int, nto : int) -> bool:
    stack = [nfrom]
    visited = set()

    while len(stack) != 0: # Recherche tant qu'il y a encore des candidats
        node = stack.pop()
        visited.add(node)

        if node == nto:
            return True # Cas où on a trouvé nto

        for neighbour in graph[node]:
            if neighbour not in visited: # On rajoute SEULEMENT les noeuds non visités
                stack.append(neighbour)

    return False # Cas où nto n'a pas été trouvé
```

4.6 Parcours en largeur *breadth first search*

L'idée du parcours en largeur, c'est de parcourir les noeuds par distance par rapport au point de départ.

Note

Cette approche a un point fort par rapport au *DFS*, car peu importe le chemin que nous obtenons jusqu'au noeud d'arrivé, ce sera le plus court chemin.

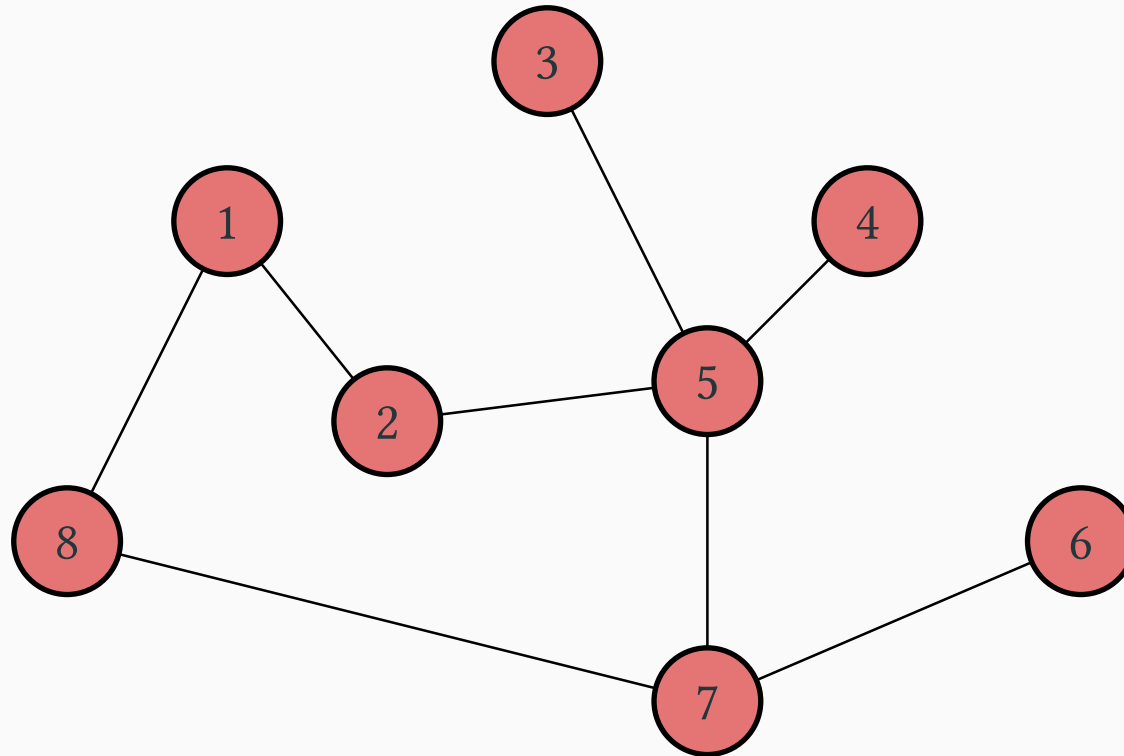
Pour appliquer cet algorithme, nous allons utiliser une file. Il suffit pour chaque noeud :

- De mettre dans une file les noeuds non visités.
- De marquer le noeud comme étant visité.

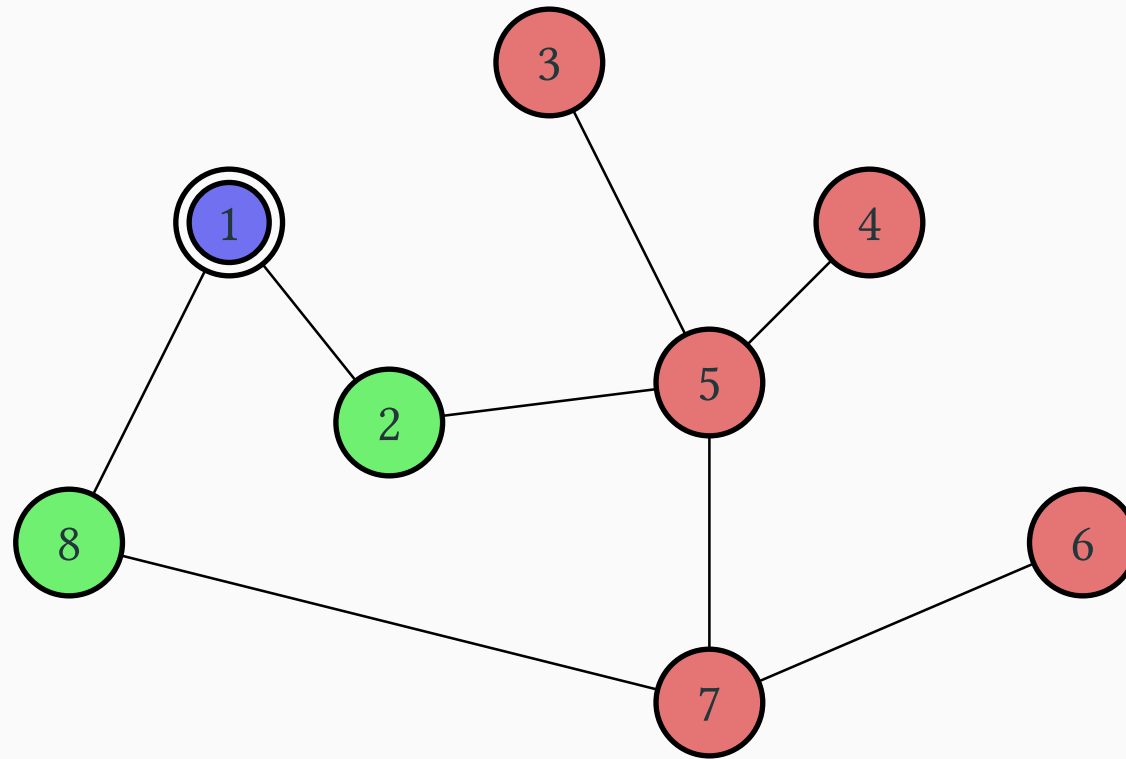
4.7 Exemple *BFS*

Les noeuds visités seront colorés en bleu, et le noeud où nous nous trouvons aura une double bordure. De plus les noeuds du prochain niveau seront en vert.

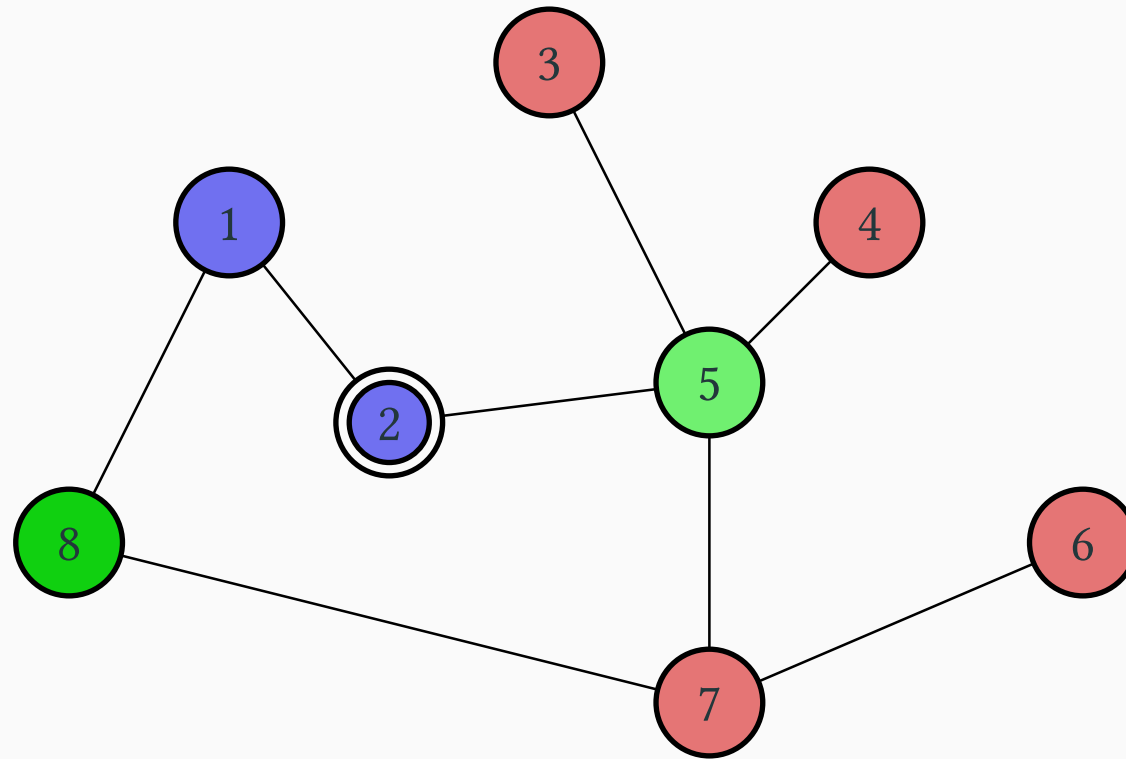
On souhaite aller du noeud 1 au noeud 6.



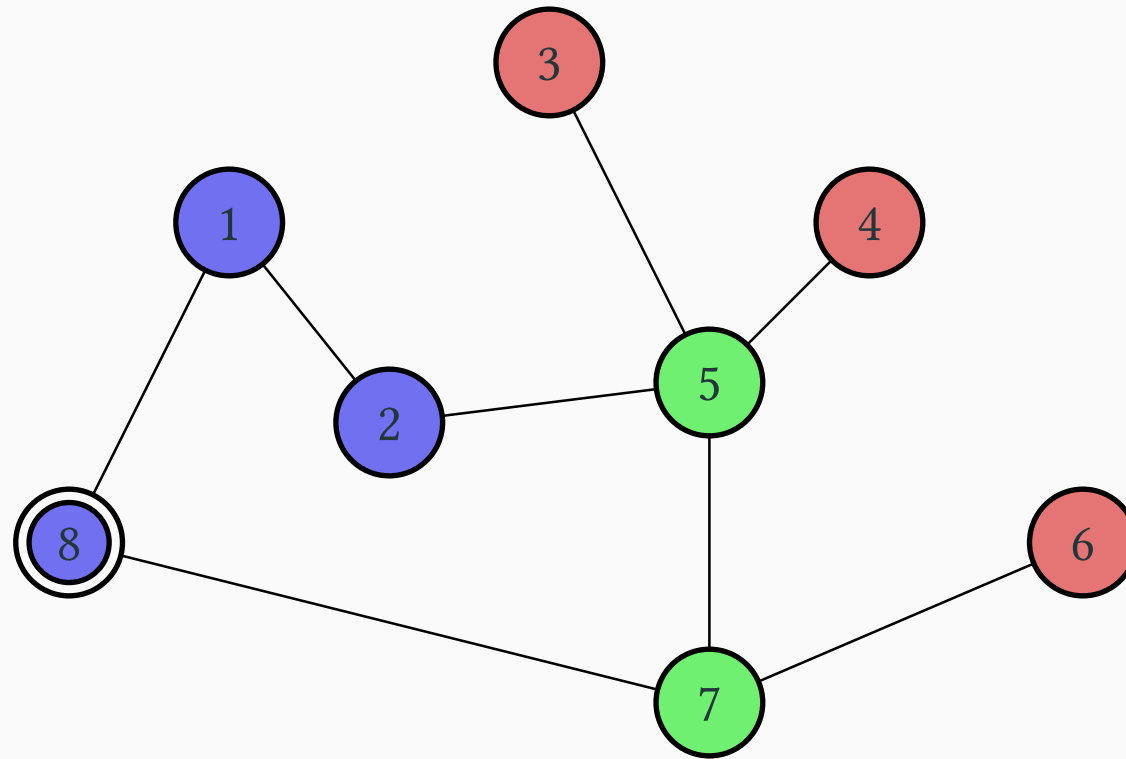
4.7 Exemple *BFS*



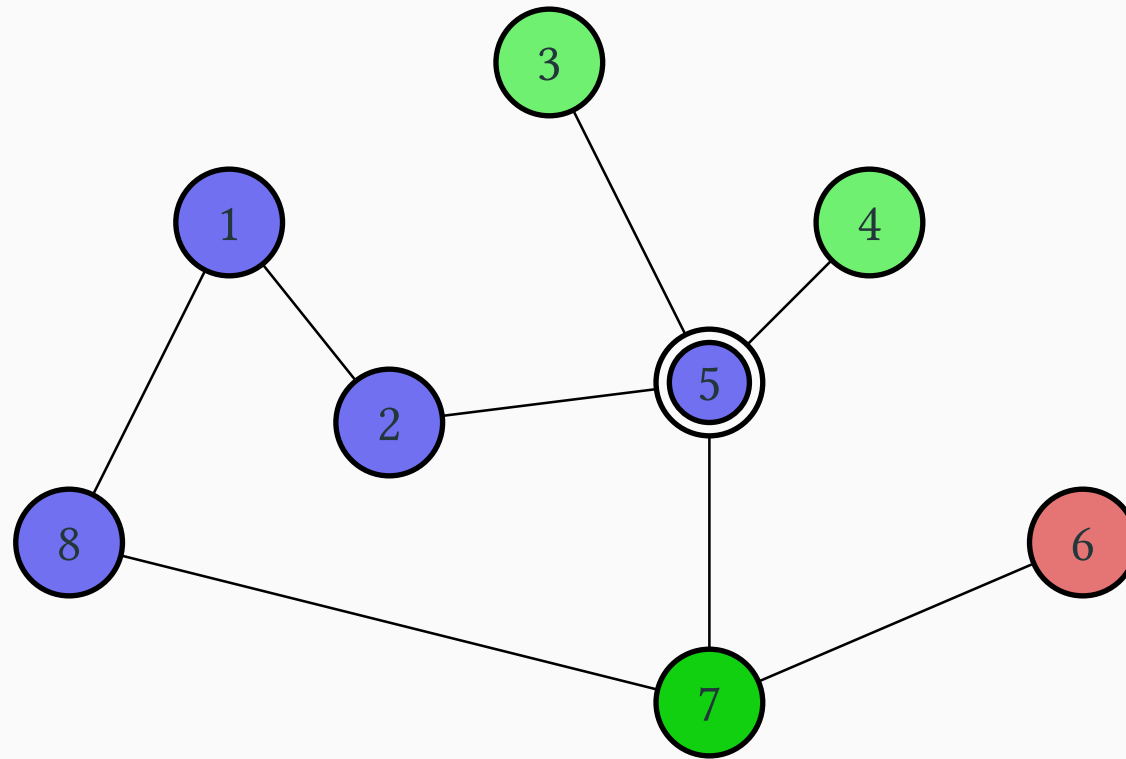
4.7 Exemple *BFS*



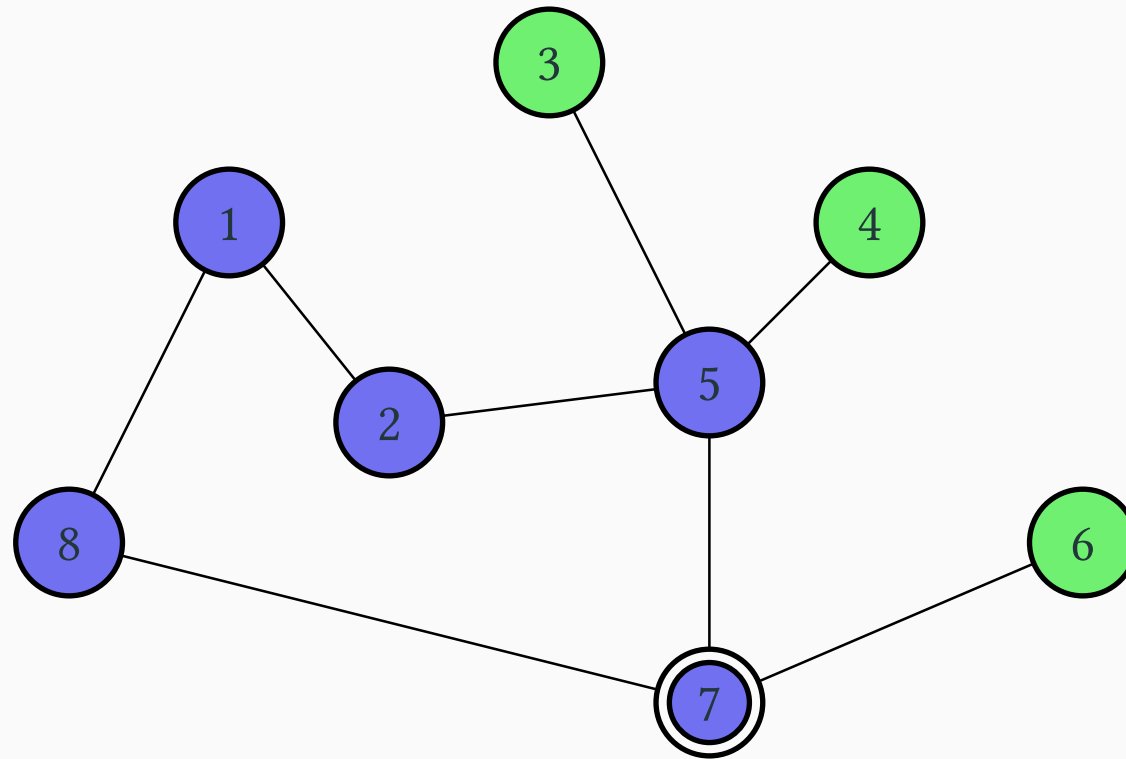
4.7 Exemple *BFS*



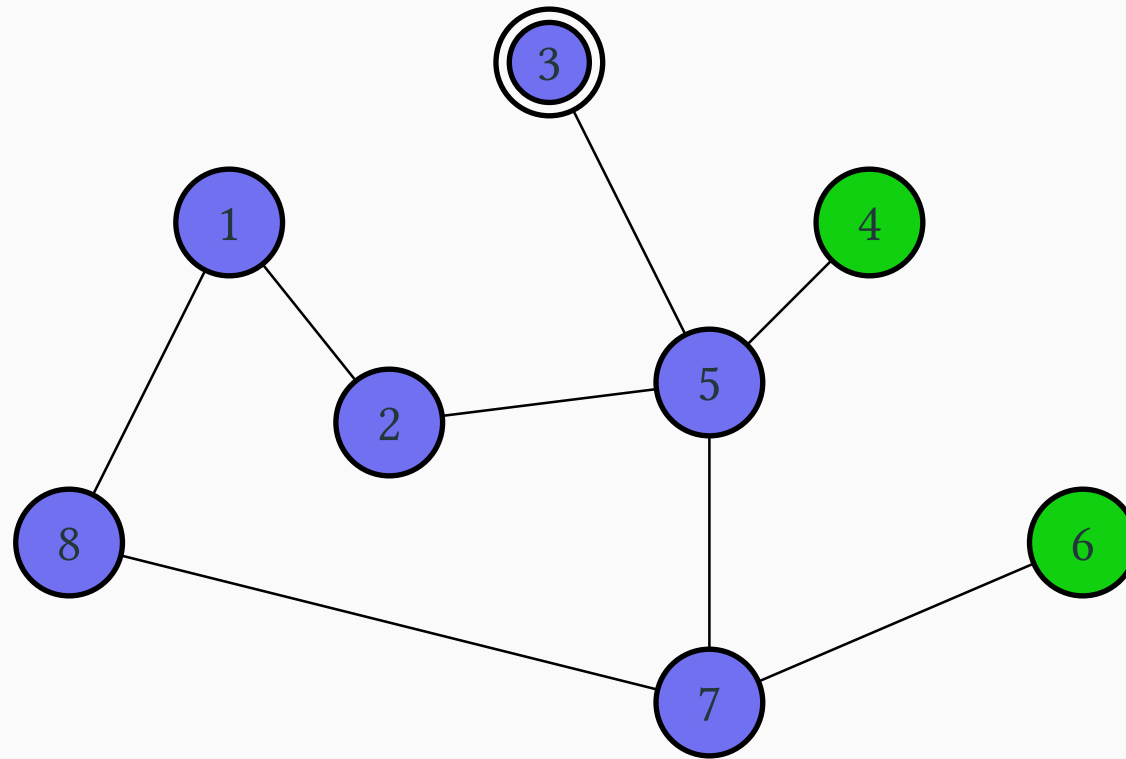
4.7 Exemple *BFS*



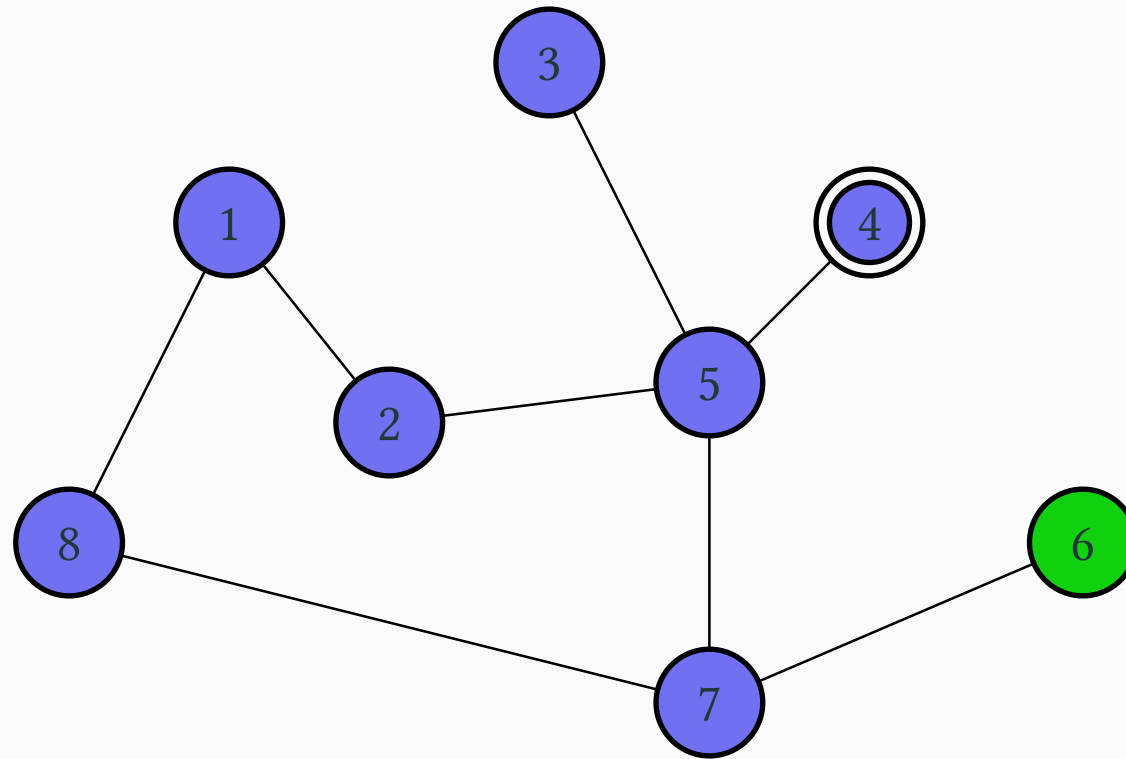
4.7 Exemple *BFS*



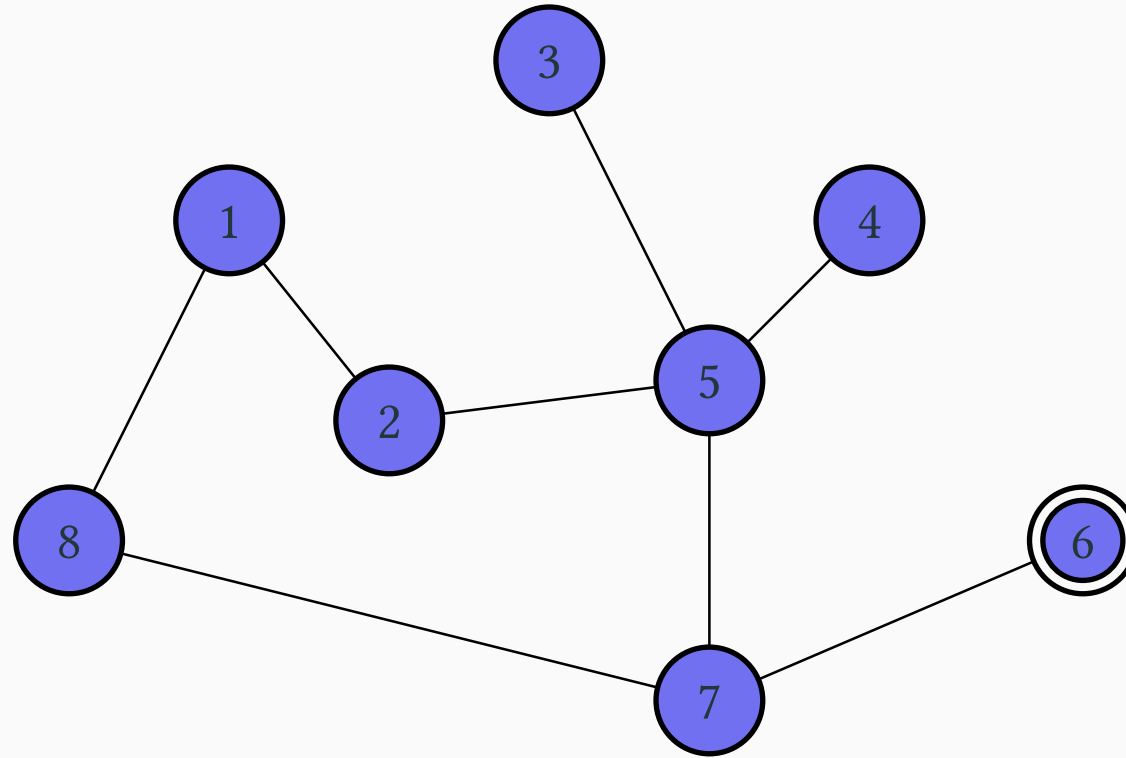
4.7 Exemple *BFS*



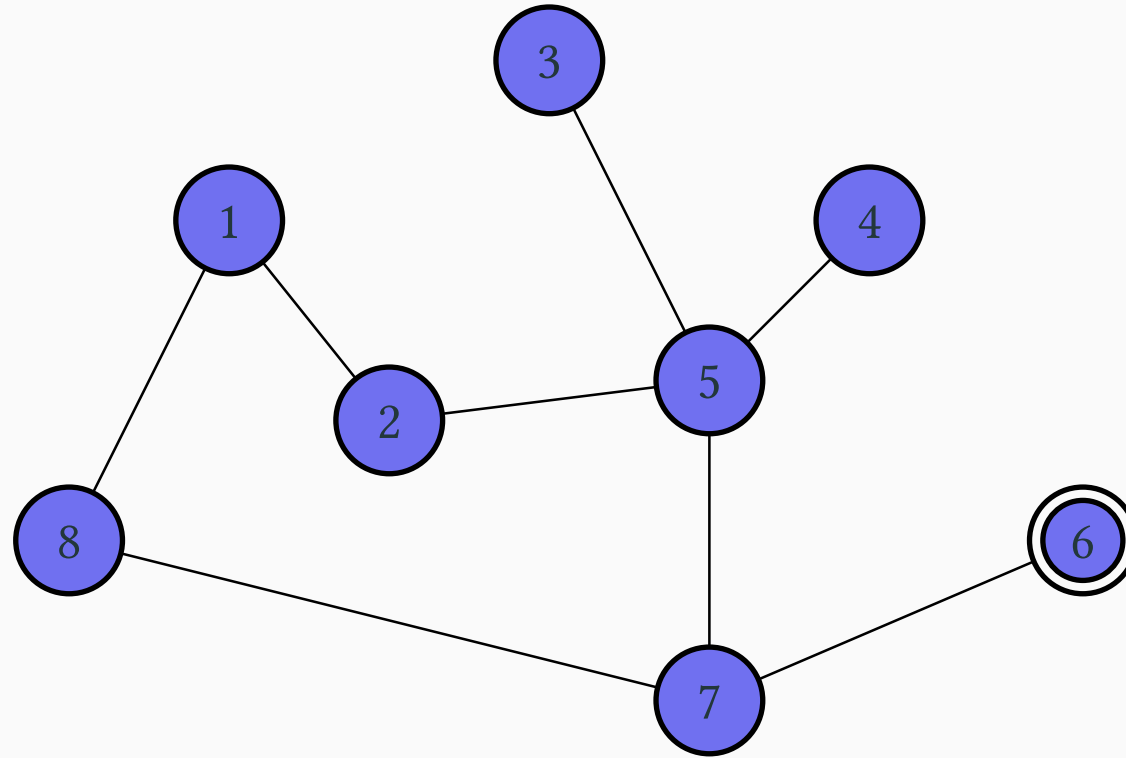
4.7 Exemple *BFS*



4.8 Exemple *BFS*



4.8 Exemple *BFS*



Nous avons trouver le noeud 6, il existe bien un chemin entre 1 et 6, et il est de longueur minimal !

4.9 Implémentation

L'implémentation ne diffère pas beaucoup de celle de *DFS*, il suffit juste de changer la pile par une file.

Voici un exemple en Python :

```
from collections import deque # Double ended queue
def bfs(graph : list[list[int]], nfrom : int, nto : int) -> bool:
    queue = deque([nfrom])
    visited = set()

    while len(queue) != 0: # Recherche tant qu'il y a encore des candidats
        node = queue.pop()
        visited.add(node)

        if node == nto:
            return True # Cas où on a trouvé nto

        for neighbour in graph[node]:
            if neighbour not in visited: # On rajoute SEULEMENT les noeuds non visités
                queue.appendleft(neighbour)

    return False # Cas où nto n'a pas été trouvé
```

4.10 Usages et informations complémentaires

Complexité de *BFS* et *DFS*

Pour chacun des algorithmes de parcours, les noeuds et arêtes sont visités une seule et unique fois. Ainsi, la complexité de ces algorithmes est $\mathcal{O}(|E| + |V|)$.

① Usage du *BFS*

Ses usages en pratique sont très fréquents pour :

- Trouver le plus court chemin (en termes d'arêtes) entre 2 noeuds.
- Trouver les composantes connexes d'un graphe.
- Trouver le plus petit cycle dans un graphe (par coloration des noeuds en 2 couleurs).

① Usage du *DFS*

Ses usages en pratique sont moins fréquents que le *BFS*, mais permet de :

- Trouver le premier chemin lexicographique entre 2 noeuds.
- Vérifier si un graphe est acyclique.
- Trouver l'ancêtre commun entre 2 noeuds le plus bas dans un arbre.

5. Crédits

Auteurs et images : Bilel MAHDJOUR

Pour le **Bureau des technologies** de **Polytech Nice Sophia**

6. Sources et ressources

- [CSES CPH book](#)
- [CP algorithms](#)
- [Stacks](#)
- [Queues \(deque\)](#)